A ZERO-TO-DEEP GUIDE

Transformers, Explained From Zero

How machines learned to read — the architecture behind GPT, Claude, Gemini, and almost every modern AI model, told in plain language.

No prior ML knowledge needed

Diagrams for every concept

Interview-ready

◎ marks interview-favorite spots · ✦ marks what matters most in today's AI era

TABLE OF CONTENTS

CHAPTER 00

Why This Actually Matters

Before the theory – a 2-minute case for why you should care

Every time you talk to ChatGPT, Claude, Gemini, or Copilot, you're talking to a **Transformer**. It's not one product — it's an *architecture*, a blueprint for building neural networks, invented in a 2017 paper called *"Attention Is All You Need."* Since then, it has quietly become the single most important idea in modern AI.

Text generation, translation, code writing, image generation (yes, even images — Stable Diffusion and Sora use Transformer-like components), protein folding (AlphaFold), speech recognition — almost all of it now runs on some flavor of this one architecture. Learning it isn't learning "one more model." It's learning the shared skeleton underneath nearly everything currently called "AI."

✦ WHY IT MATTERS TODAY

You don't need to be a machine learning engineer for this to be useful. Product managers, engineers integrating AI APIs, students, and interview candidates across the board are expected to have at least a working mental model of "what is actually happening inside the model." This guide builds exactly that — without requiring you to know calculus or write a line of code.

→ How to read this guide

We go in order, start to finish, and we never introduce a term without explaining it first. Every complex term gets its **own boxed-out section** like the one below, so you can also use this later as a lookup reference.

COMPLEX TERM, EXPLAINED

Example: Neural Network

In plain words: a very large mathematical function, built from simple pieces, that learns patterns from data by adjusting millions (or billions) of internal numbers called "weights."

You'll see boxes like this throughout the guide for every term that isn't everyday English — things like "embedding," "self-attention," or "softmax." Read them once, and you'll be able to hold a real conversation about how modern AI actually works.

Watch for two kinds of markers as you read:

◎ INTERVIEW SPOTLIGHT

Marks a point that interviewers love to probe — for ML engineering, data science, or even general software roles where "explain how an LLM works" has become a common question.

✦ WHY IT MATTERS TODAY

Marks a point that connects directly to tools you already use — ChatGPT, Claude, GitHub Copilot, Midjourney, and so on.

CHAPTER 01

The Big Picture

What problem are we even solving, and what came before?

1.1 The problem: understanding sequences

Language is a **sequence** — words arrive one after another, and order changes meaning entirely ("dog bites man" vs. "man bites dog"). Any system that reads, writes, or translates language has to somehow understand not just individual words, but how every word relates to every other word around it.

That's the entire problem a Transformer is built to solve: *given a sequence of words, figure out how they relate to each other, and use that to understand or predict language.*

1.2 Life before Transformers

Before 2017, the standard tool for sequences was the **RNN** (Recurrent Neural Network), often in an upgraded form called **LSTM** (Long Short-Term Memory).

COMPLEX TERM, EXPLAINED

RNN / LSTM

In plain words: a model that reads a sentence one word at a time, carrying a "memory summary" forward as it goes – like reading a book while jotting a running mental summary, one word at a time, left to right.

The problem: this is **sequential by design**. Word 5 can't be processed until word 4 is done, which is done only after word 3, and so on. Two costs follow directly: it's slow to train on long text (no shortcuts, no parallel work), and the "memory summary" tends to forget things from far earlier in a long sentence — much like you might forget the start of a very long sentence by the time you reach the end.

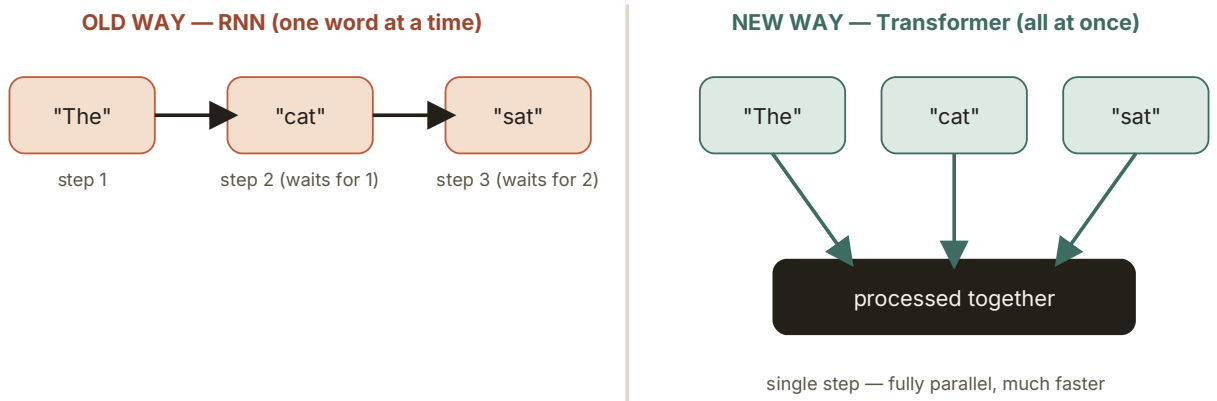


Fig 1.1 — The core shift: sequential reading vs. parallel reading

INTERVIEW SPOTLIGHT

"Why did Transformers replace RNNs?" is one of the most common opening questions in ML interviews. The two-part answer they want: **(1) parallelization** — Transformers process every word simultaneously instead of one-by-one, making training dramatically faster on modern hardware (GPUs/TPUs), and **(2) long-range dependencies** — attention lets any word directly connect to any other word, no matter how far apart, without the "forgetting" problem RNNs have.

1.3 The one-sentence idea

THE CORE IDEA

Instead of reading a sentence word-by-word and carrying a fading memory forward, a Transformer lets **every word look directly at every other word at the same time**, and decide for itself which ones matter most. That mechanism is called **attention**, and it's the single most important idea in this entire guide.

Everything else in the Transformer — embeddings, positional encoding, feed-forward layers, normalization — exists to support that one mechanism. Get comfortable with the idea of attention, and the rest of the architecture is just plumbing around it.

1.4 The zoomed-out map

Here is the entire journey a sentence takes through a Transformer, zoomed all the way out. We will expand every single box below into its own chapter.

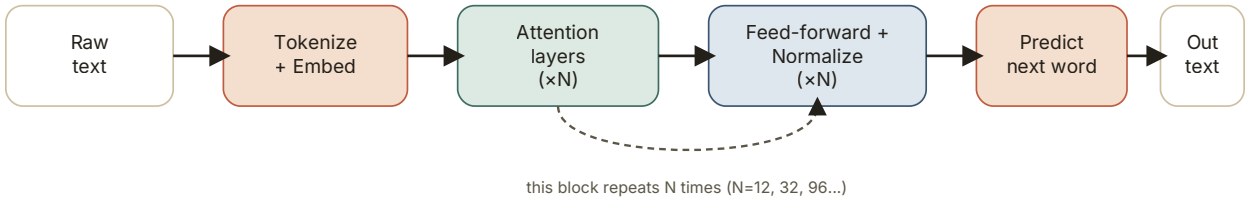


Fig 1.2 — The full journey, zoomed out. Chapters 2 through 6 expand each box.

✦ WHY IT MATTERS TODAY

That "predict next word" box is not a simplification — it is *literally* what GPT, Claude, and every similar model do, one word (technically, one "token") at a time. An entire essay, poem, or piece of code is produced by repeating that single box over and over, each time feeding the newly produced word back in as part of the input for the next one. This is called **autoregressive generation**, and it's why LLMs sometimes "talk themselves into" a wrong answer — each new word is chosen based only on what came before it, with no ability to plan the whole sentence in advance.

CHAPTER 02

Turning Words Into Numbers

Computers don't understand words. Here's how text becomes math.

A neural network is, underneath everything, a giant calculator — it only understands numbers. So before any "understanding" can happen, a sentence has to be converted into numbers, without losing its meaning. This happens in three steps, and each one deserves its own explanation.

2.1 Step one: breaking text into pieces

COMPLEX TERM, EXPLAINED

Tokenization

In plain words: chopping a sentence into small chunks – called "tokens" – the way you might chop vegetables before cooking. A token is often a whole word, but not always.

You might expect tokens to simply be words. Modern models actually use **subword tokenization** (a common method is called BPE — Byte Pair Encoding): common words stay whole ("the," "cat"), but rarer or longer words get split into pieces ("tokenization" might become "token" + "ization"). This gives the model a manageable, fixed-size vocabulary (commonly 30,000 – 100,000 possible tokens) while still being able to represent literally any word, even ones it has never seen, by falling back to smaller pieces.

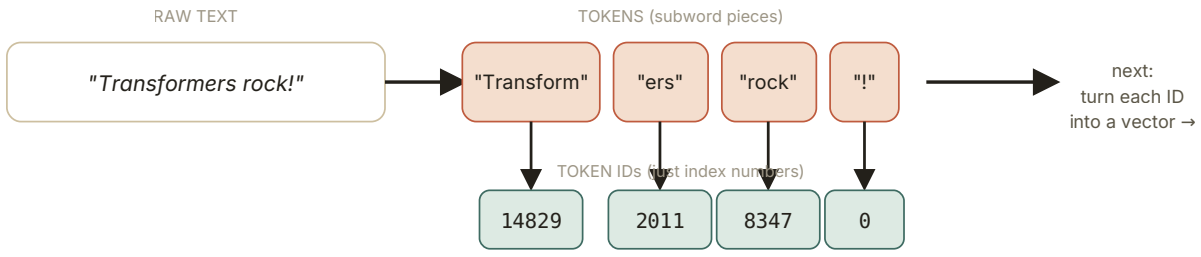


Fig 2.1 — Text is chopped into subword tokens, then mapped to plain index numbers via a fixed vocabulary lookup table

✦ **WHY IT MATTERS TODAY**

Every AI API — OpenAI, Anthropic's Claude API, Google's Gemini API — bills you **per token**, not per word or per character. This is exactly why: the model's real "unit of work" is the token. It's also why a word in a less common language, or an unusual name, can quietly cost more than a common English word — it may take several tokens to represent instead of one.

◎ **INTERVIEW SPOTLIGHT**

"Why not just tokenize by whole word?" Because vocabulary would explode (every typo, name, and made-up word needs its own slot), and any word not in the training vocabulary becomes an unrepresentable "unknown" token. Subword tokenization guarantees **any** string of text can be represented, since in the worst case, it falls back to individual characters.

2.2 Step two: giving each token meaning

A token ID like `14829` is just an arbitrary index — it carries no meaning by itself. Token 14829 being "next to" token 14830 in that list tells you nothing about whether the two words are related. We need something better: a representation where *similar meaning gives you a similar representation.*

COMPLEX TERM, EXPLAINED

Embedding

In plain words: a list of numbers (a "vector") that represents a word's meaning as a point in space – like GPS coordinates, but for meaning instead of location.

Each token gets converted into a vector of, typically, several hundred to over ten thousand numbers. The model learns these numbers during training such that words used in similar contexts end up **close together** in this space. "Cat" and "dog" end up near each other; "cat" and "bicycle" end up far apart. This space even captures relationships: the classic demonstration is that the vector math `king - man + woman` lands you very close to the vector for `queen`.

✦ **WHY IT MATTERS TODAY**

Embeddings are the backbone of **semantic search** and **RAG (Retrieval-Augmented Generation)** — the technique most "chat with your documents" products use. Your document gets converted into embedding vectors, your question gets converted into an embedding vector, and the system finds the document chunks whose vectors are mathematically closest to your question's vector. This is also exactly how modern recommendation systems and image search work.

2.3 Step three: remembering word order

Here's a subtle but critical problem: the attention mechanism we're about to meet in Chapter 3 looks at all words *at once*, with no built-in sense of which word came first. Left completely on its own, a Transformer would see "the dog bit the man" and "the man bit the dog" as containing the exact same information — just an unordered bag of words.

COMPLEX TERM, EXPLAINED

Positional Encoding

In plain words: a second vector added on top of the embedding, one per token, whose entire job is to encode "I am word number 1," "I am word number 2," and so on – like stamping a timestamp on every word.

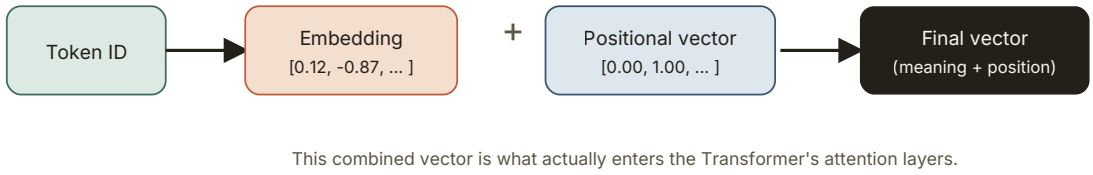
The original 2017 paper used a fixed mathematical pattern built from sine and cosine waves at different frequencies (you don't need the formula — just know it's a deliberately-designed, unique "fingerprint" for each position). This fingerprint vector is added directly on top of the word's embedding vector before anything else happens, so from that point on, every vector carries both *what* the word means and *where* it sits in the sentence.

◎ INTERVIEW SPOTLIGHT

"Why do Transformers need positional encoding but RNNs don't?" — because RNNs process words one at a time in order, so position is implicit in the process itself. Transformers process every word simultaneously (that's the whole point — see Chapter 1), which means position has to be explicitly injected as information, or it's lost entirely. This is a favorite follow-up question after "explain self-attention."

✦ WHY IT MATTERS TODAY

Most current large language models (LLaMA, Mistral, and many others) no longer use the original sine/ cosine version — they use a newer technique called **RoPE (Rotary Positional Embeddings)**, which encodes position by mathematically rotating the vectors. You don't need the math, but knowing the name "RoPE" and that "positional encoding has evolved past the original design" is a strong, current signal in any technical conversation about LLMs.



This combined vector is what actually enters the Transformer's attention layers.

Fig 2.2 — Every token's final representation is its meaning vector plus its position vector

With every word now represented as a single rich vector — carrying both meaning and position — we're finally ready for the main event: attention.

CHAPTER 03

Attention – The Heart of the Machine

The single mechanism that makes everything else possible

3.1 The intuition, before any math

ANALOGY: THE COCKTAIL PARTY

Imagine standing in a noisy room full of conversations. Somehow, your brain can tune into the one conversation that matters to you right now, while quieting the rest – and a moment later, tune into a different one if someone across the room says your name. That's **attention**: dynamically deciding, from moment to moment, what to focus on and how strongly.

Now apply that to reading. Take the sentence: *"The animal didn't cross the street because it was too tired."* What does "it" refer to? You instantly know it means "the animal," not "the street" — because you weigh the relevance of every other word in the sentence when interpreting "it." A Transformer needs to do the exact same thing, for every single word, and it does this using a mechanism called **self-attention**.

3.2 The three roles: Query, Key, Value

To let words "look at" each other and measure relevance, every token's vector gets projected into three different versions of itself, each with a specific job. This is easiest to understand with an analogy before we touch a single diagram.

ANALOGY: A LIBRARY SEARCH

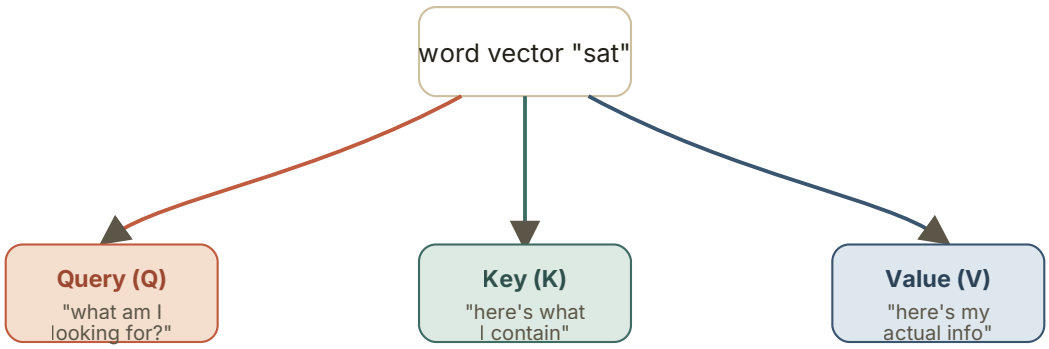
Think of a library. You walk in with a **Query** – what you're looking for ("books about volcanoes"). Every book on the shelf has a **Key** – like a catalog tag summarizing what it's about. You compare your query against every book's key to find the best matches. Once you've found the right books, what you actually walk out with is their **Value** – the actual content inside.

COMPLEX TERM, EXPLAINED

Query, Key, and Value (Q, K, V)

In plain words: three different "lenses" applied to the same word vector, each created by multiplying it against a separate learned matrix of weights.

Concretely: every input vector is multiplied by three different weight matrices (`W_Q`, `W_K`, `W_V`) — all learned during training — producing a Query vector, a Key vector, and a Value vector for that word. **Every single word produces all three**, simultaneously. A word's Query asks "what am I looking for?"; its Key advertises "here's what I contain, for anyone searching"; its Value carries "here's the actual information I'll hand over if picked."



Three separate learned matrices (W_Q, W_K, W_V) create three vectors from the same input.
Every word in the sentence does this — not just one.

Fig 3.1 — Each word generates its own Query, Key, and Value vector

3.3 Self-attention, step by step

Now for the actual mechanism. Let's trace it through a tiny 3-word sentence: *"The cat sat."* We'll compute the attention output for the word **"sat."**

Step 1 — Compare "sat"'s Query against everyone's Key

Take the Query vector for "sat," and compare it — using a dot product, a simple similarity measurement — against the Key vector of every word in the sentence, including itself. This produces one raw **score** per word: a number that's higher when the vectors point in a more similar direction.

Step 2 — Scale, then turn scores into percentages

The raw scores are first divided by a scaling factor (the square root of the Key vector's dimension — more on why in the box below), then passed through a function called **softmax**, which turns any list

of numbers into positive values that all add up to 100%. Now instead of raw scores, we have clean **attention weights** — e.g. "sat" might attend 70% to "cat," 20% to "sat" itself, and 10% to "the."

COMPLEX TERM, EXPLAINED

Softmax

In plain words: a function that takes a list of numbers – any numbers, positive or negative – and rescales them into a list of percentages that add up to exactly 100%, while keeping the biggest number the most dominant.

It's how the model turns "raw relevance scores" into something usable as a weighting scheme: clean, comparable, always-positive values that behave like a probability distribution.

COMPLEX TERM, EXPLAINED

Scaled Dot-Product (the "scaled" in Scaled Dot-Product Attention)

In plain words: dividing the similarity scores by a fixed number before the softmax step, purely to keep the numbers in a stable, well-behaved range.

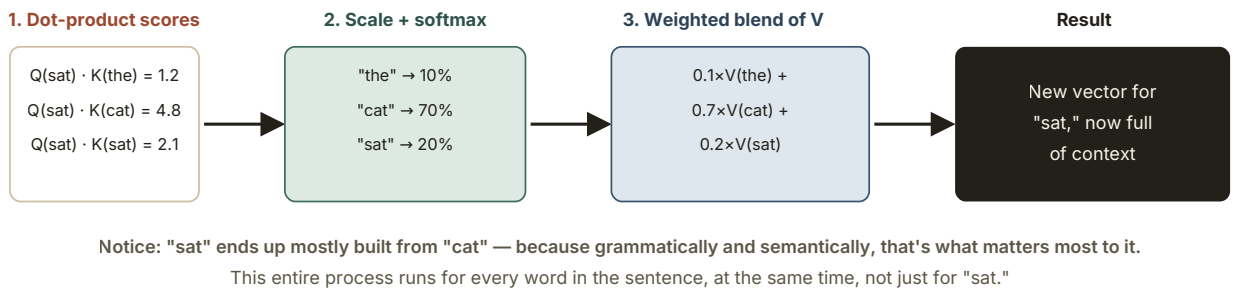
Without this, as vectors get longer (more dimensions), raw dot-product scores tend to grow very large, which pushes softmax into an extreme state (almost all weight on one word, near-zero elsewhere) — making training unstable. Dividing by the square root of the Key dimension keeps the scores in a sane range regardless of vector size.

Step 3 — Blend the Values using those weights

Finally, multiply every word's **Value** vector by its attention weight, and add them all up. The result is a brand-new vector for "sat" — no longer just "sat" in isolation, but "sat," enriched with a weighted blend of context from every relevant word around it.

THE ENTIRE MECHANISM, AS ONE FORMULA

$$\text{Attention}(Q, K, V) = \text{softmax}\left(Q \cdot K^T / \sqrt{d_k} \right) \cdot V$$



Notice: "sat" ends up mostly built from "cat" — because grammatically and semantically, that's what matters most to it.
This entire process runs for every word in the sentence, at the same time, not just for "sat."

Fig 3.2 — Self-attention for the word "sat," worked through numerically

INTERVIEW SPOTLIGHT

Being able to write and explain $\texttt{Attention(Q,K,V) = softmax(QK}^T/\sqrt{d_k}\texttt{)V}$ from memory, and explain *why* each piece exists (dot product = similarity, scale = stability, softmax = normalize into weights, multiply by V = blend actual content) is very possibly the single most common "explain this" question in ML interviews today. It comes up even in interviews that aren't specifically for ML roles.

3.4 One head isn't enough

Here's a limitation: a single attention calculation captures *one kind* of relationship at a time — say, matching subjects to verbs. But language has many simultaneous kinds of relationships: grammar, meaning, tone, coreference (like "it" referring back to "animal"), and more, all at once. One single attention pass can't specialize in all of them.

COMPLEX TERM, EXPLAINED

Multi-Head Attention

In plain words: instead of doing attention once, do it several times in parallel, with each "head" allowed to learn to focus on a different kind of relationship – then combine all their findings.

Practically: the Query, Key, and Value vectors are split into several smaller chunks (commonly 8, 12, 16, or more "heads"). Each head independently runs the exact same attention process on its own chunk. One head might end up specializing in grammatical structure, another in tracking pronouns, another in factual associations — nobody assigns these roles; they simply emerge from training. All the heads' outputs are then concatenated back together and passed through one more learned transformation to produce the final output.

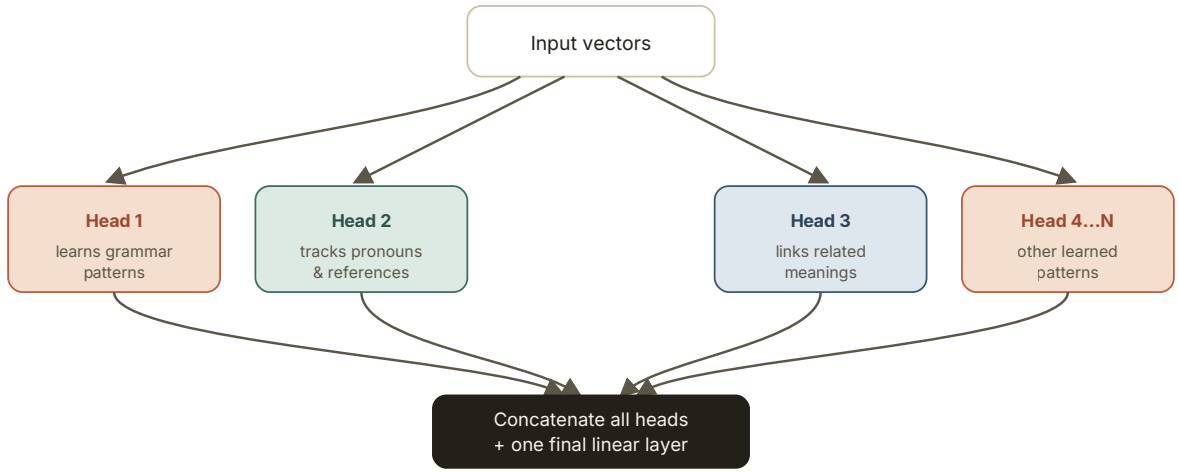


Fig 3.3 — Multiple attention heads run in parallel, each free to specialize, then get merged back together

✦ WHY IT MATTERS TODAY

When researchers talk about "interpretability" — trying to understand what's actually happening inside models like Claude or GPT — a major line of work involves poking at individual attention heads to see what they've learned to specialize in. Anthropic's own interpretability research has found heads and circuits responsible for surprisingly specific behaviors, which is a big part of how the field is trying to make these systems more transparent and trustworthy.

◎ INTERVIEW SPOTLIGHT

"Why split into multiple heads instead of just using one larger attention calculation?" Splitting into smaller parallel heads lets the model learn **several different types of relationships simultaneously**, rather than being forced to average everything into one generic notion of relevance. It's often compared to having several specialists look at a problem instead of one generalist.

CHAPTER 04

The Supporting Cast

Attention gets the spotlight, but it can't work alone

Attention is the star of the show, but three quieter components make it actually work in practice at scale. None of them are conceptually hard — think of this chapter as short, necessary stagehands work before we assemble the full architecture in Chapter 5.

4.1 Giving each word room to "think"

After attention blends context from surrounding words, the model needs a step to actually *process* that new information — some further computation on each word's vector, independently.

COMPLEX TERM, EXPLAINED

Feed-Forward Network (FFN)

In plain words: a small, standard neural network applied identically to every word's vector, one at a time – like giving each word a private moment to digest what attention just handed it.

Structurally, it's simple: two layers with a non-linear function in between (commonly ReLU or GELU, which you don't need to memorize — just know they let the network represent more complex, non-straight- line relationships than plain multiplication could). It typically expands the vector to a much larger size internally, then compresses it back down. Unlike attention, the feed-forward step does **not** let words look at each other — it works on each word's vector independently and identically.

INTERVIEW SPOTLIGHT

A clean way to describe the division of labor: **"attention moves information between words; the feed-forward network processes information within a word."** Interviewers like hearing this distinction stated explicitly — it shows you understand the architecture isn't just "one big attention blob."

4.2 Not losing the original signal

As information passes through many stacked layers (real models stack dozens, sometimes over a hundred), there's a real risk that the original signal gets diluted or that gradients (the signal used to update the model during training) struggle to flow backward through so many layers.

COMPLEX TERM, EXPLAINED

Residual Connection (a.k.a. Skip Connection)

In plain words: instead of fully replacing a word's vector with the output of a layer, add the layer's output on top of the original input – keeping a direct, unmodified path alongside every transformation.

Picture a highway with local exits: the main highway (the original vector) keeps going in a straight line, and each layer is an off-ramp that computes something useful and merges its contribution back onto the highway, rather than closing the highway and rerouting everything through itself. This one small architectural trick is a huge reason very deep networks (Transformers included) are trainable at all.

4.3 Keeping the numbers well-behaved

COMPLEX TERM, EXPLAINED

Layer Normalization

In plain words: after each step, rescale every word's vector so its numbers sit in a consistent, stable range – like a sound engineer keeping every microphone at a similar volume level instead of letting one channel drift off into distortion.

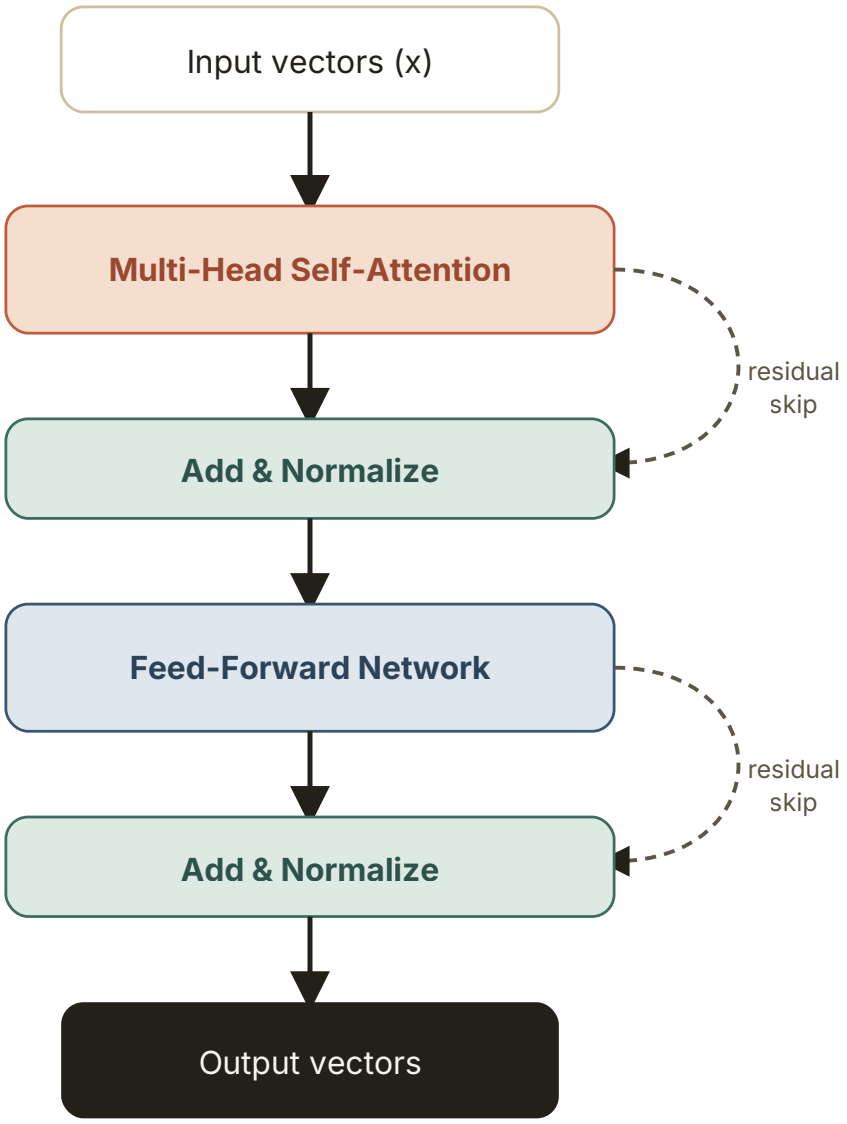
Without this, as vectors pass through many layers, their scale can drift unpredictably — some numbers ballooning, others shrinking toward zero — which makes training slow or unstable. Layer normalization is applied around both the attention step and the feed-forward step, working together with residual connections to keep very deep networks trainable.

✦ WHY IT MATTERS TODAY

Modern large models often use a variant called **RMSNorm** instead of the original Layer Normalization, and place normalization *before* each sub-layer rather than after (called "pre-norm") for better training stability at massive scale. If you read model papers or technical reports (LLaMA, Claude's, or others), these small naming details — RMSNorm, pre-norm — are exactly the kind of detail that separates a surface-level read from a genuine understanding of what changed.

4.4 Putting the three together: one full block

These three components combine with multi-head attention into a single, repeatable unit — often called a **Transformer block** or **Transformer layer**. A real model stacks many of these blocks on top of each other (the "×N" you saw back in Fig 1.2).



One "Transformer block." Stack this N times to build the full model.

Fig 4.1 — One complete Transformer block: attention, residuals, normalization, and the feed-forward network working together

INTERVIEW SPOTLIGHT

Being able to draw Fig 4.1 from memory on a whiteboard — attention, add & norm, feed-forward, add & norm, in the right order, with the residual skips in the right place — is a extremely common "walk me through the architecture" ask. Practice sketching it a few times; the shape matters more than the exact wording.

CHAPTER 05

Assembling the Full Architecture

Stacking blocks into encoders, decoders, and full models

5.1 Two kinds of stacks: Encoder and Decoder

The original 2017 Transformer was built for machine translation, and it used **two** stacks of blocks working together: an Encoder stack that reads and understands the input sentence, and a Decoder stack that generates the output sentence, one word at a time, using what the Encoder understood.

COMPLEX TERM, EXPLAINED

Encoder

In plain words: a stack of Transformer blocks (Chapter 4's Fig 4.1) whose job is to read the entire input sentence and build a rich, context-aware representation of it – no restrictions on what it can look at.

Every word in the encoder can attend to every other word, including words that come after it. This makes sense for understanding a complete input sentence you already have in full — there's no reason to hide the second half of the sentence from the first half.

COMPLEX TERM, EXPLAINED

Decoder

In plain words: a stack of Transformer blocks whose job is to generate the output, one word at a time, using both what it has generated so far and what the Encoder understood about the input.

Crucially, the decoder is not allowed to peek at future words it hasn't generated yet — that would be cheating (and impossible at real use-time, since those words don't exist yet). This restriction is enforced by **masking**, explained next.

5.2 Stopping the model from cheating

COMPLEX TERM, EXPLAINED

Masked Self-Attention (Causal Masking)

In plain words: when computing attention inside the decoder, forcibly block each word from seeing any word that comes after it – as if the rest of the sentence is physically covered up.

Technically, this is done by setting the attention scores for "future" positions to negative infinity before the softmax step, which makes their resulting weight exactly zero. The word "sat" can attend to "the" and "cat" (and itself), but not to any word appearing after it. This is what makes the decoder **autoregressive** — generating strictly left-to-right, one token justified only by the tokens before it.

◎ INTERVIEW SPOTLIGHT

"Why does the decoder need masking but the encoder doesn't?" — because the encoder sees a complete, already-known input sentence (no risk of "cheating"), while the decoder is generating output sequentially and must not have access to tokens it hasn't produced yet — otherwise, it would learn to trivially "cheat" during training by copying the answer, and would break entirely at real inference time when future tokens genuinely don't exist yet.

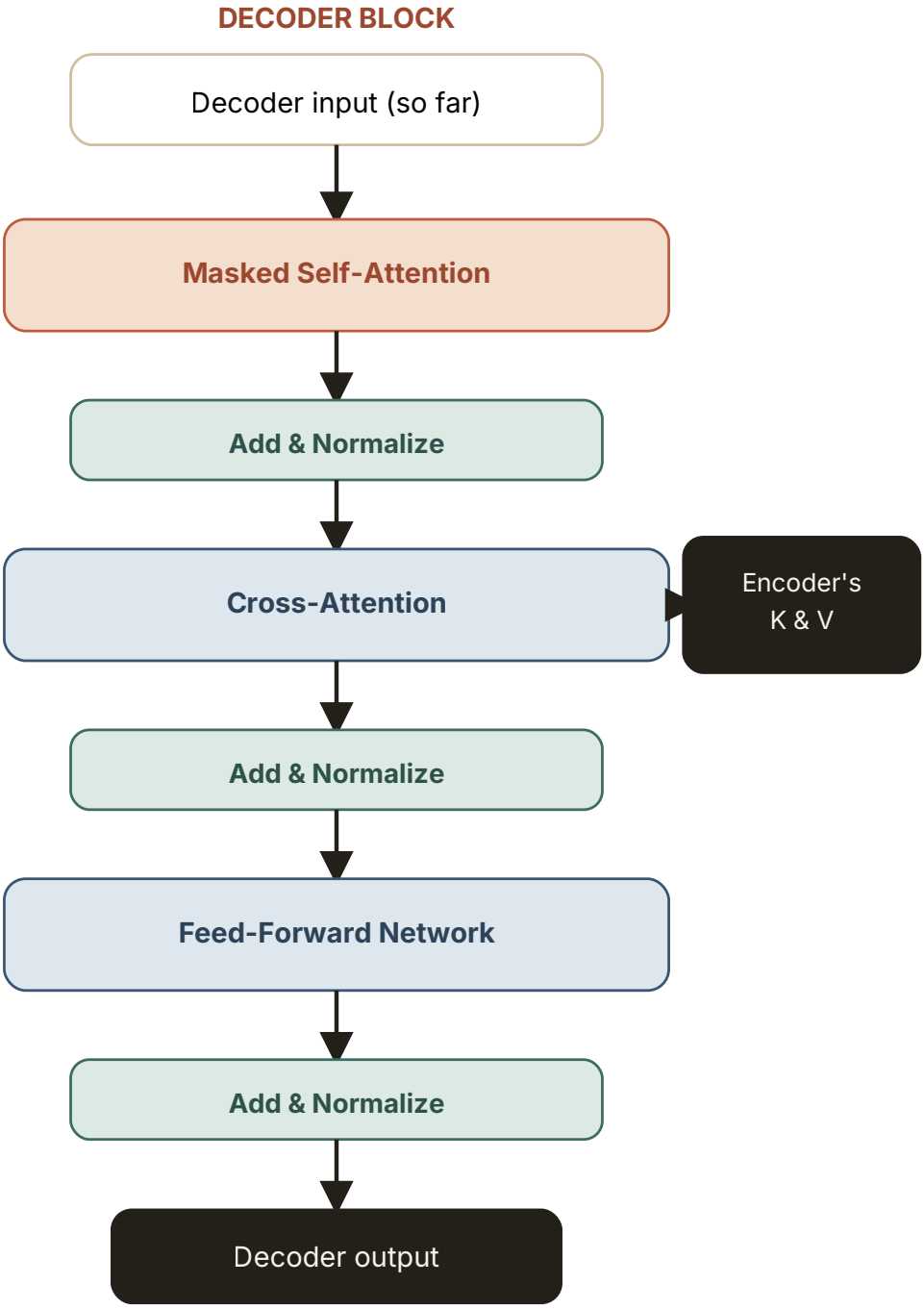
5.3 Connecting the two stacks

COMPLEX TERM, EXPLAINED

Cross-Attention

In plain words: a second type of attention inside each decoder block where the decoder's Queries meet the encoder's Keys and Values – letting every output word directly consult the input sentence while being generated.

This is exactly the Query/Key/Value mechanism from Chapter 3, just with a twist: the Queries come from the decoder (representing "what should I generate next, and what do I need to check against the source sentence to get it right?"), while the Keys and Values come from the encoder's final output (the fully understood input sentence). It's the direct bridge between "understanding the input" and "producing the output" — critical for tasks like translation, where every output word needs to be traceable back to something in the input.



Same skeleton as the encoder block (Fig 4.1), plus one extra ingredient: cross-attention, which links back to everything the encoder understood.

Fig 5.1 — Inside one decoder block: masked self-attention, then cross-attention into the encoder, then feed-forward

5.4 The complete original architecture

Zoom all the way out, and here is the full picture: an Encoder stack of N blocks on the left, a Decoder stack of N blocks on the right, connected by cross-attention, with the input sentence flowing in at the bottom left and the generated output flowing in (one step behind itself) at the bottom right.

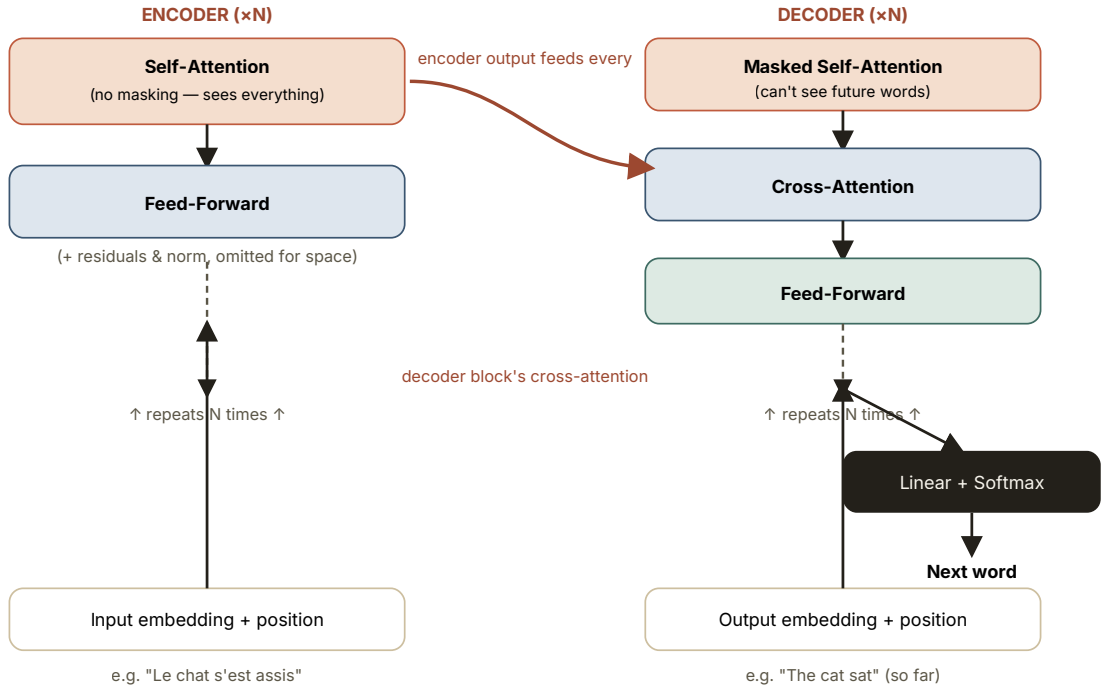


Fig 5.2 — The complete original Transformer: Encoder + Decoder, from the 2017 paper (simplified)

5.5 The three architecture families you'll actually see

Here's something the original paper's diagram doesn't tell you: **most modern models only use half of this picture.** Depending on the task, researchers dropped one of the two stacks entirely. This single fact explains a huge amount of the modern AI landscape.

FAMILY	USES	GOOD AT	EXAMPLES
Encoder-only	Just the Encoder	Understanding / classifying existing text	BERT, sentence embedding models
Decoder-only	Just the (masked) Decoder	Generating text, one token at a time	GPT models, Claude, LLaMA, Gemini
Encoder–Decoder	Both, connected by cross-attention	Transforming one sequence into another	The original Transformer, T5, most translation systems

✦ WHY IT MATTERS TODAY

This is arguably the single most practically important fact in this entire guide: **Claude, GPT, and virtually every chatbot you've used is a decoder-only model.** There is no separate encoder and no cross-attention in the sense of Fig 5.1 — there's just one stack of masked self-attention blocks, reading the entire conversation so far (your messages and its own previous replies) as one long sequence, and repeatedly predicting the next token. "Understanding your question" and "generating the answer" happen inside the exact same stack, not two separate ones.

◎ INTERVIEW SPOTLIGHT

"Is GPT an encoder or a decoder?" is a near-guaranteed question if a conversation touches LLMs at all. The correct, confident answer: **decoder-only**. A strong follow-up point to offer unprompted: BERT (encoder-only) is typically used for understanding tasks like classification or search, while GPT-style models (decoder-only) are used for open-ended generation — and this maps directly onto why BERT can't "write an essay" the way GPT can, and why GPT is comparatively less naturally suited to tasks like filling in a blank in the middle of an existing document.

CHAPTER 06

From Vectors Back To Words

Turning the model's final math back into language you can read

6.1 The last step: picking a word

After passing through every Transformer block, the model has a final, richly-informed vector for the next position. But that's still just a vector — a list of numbers. The very last step converts it into an actual probability for every single word in the model's vocabulary.

COMPLEX TERM, EXPLAINED

Output Layer (Linear + Softmax)

In plain words: one final multiplication that scores every word in the model's entire vocabulary based on the final vector, followed by the same softmax function from Chapter 3 – this time turning those scores into "here's the probability of every possible next word."

If the vocabulary has 50,000 possible tokens, this step produces 50,000 numbers, all between 0 and 1, all summing to 100%. For the phrase "The cat sat on the ___," the word "mat" might get 34%, "floor" 22%, "sofa" 9%, and so on, with tens of thousands of other words splitting the remaining tiny fractions.

6.2 Choosing from the probabilities

You might assume the model always just picks the single highest-probability word. In practice, that's a choice, not a requirement — and it's a choice that dramatically changes the model's personality.

COMPLEX TERM, EXPLAINED

Sampling (Temperature, Top-k, Top-p)

In plain words: the strategy used to pick one word out of the probability list – ranging from "always take the safest, most likely word" to "roll the dice, weighted toward more likely words, but leave room for surprise."

Temperature reshapes how sharply peaked the probabilities are before picking — low temperature (near 0) makes the model nearly always choose the top word (safe, repetitive, deterministic); high temperature flattens the odds, giving rarer words a real chance (more creative, more prone to mistakes). **Top-k** restricts the choice to only the k most likely words before sampling; **top-p** (also called nucleus sampling) instead keeps the smallest set of words whose probabilities add up to some threshold p (like 90%), which adapts better than top-k when the model is very confident vs. very unsure.

✦ WHY IT MATTERS TODAY

This is not a theoretical detail — it's a real, user-facing setting. Every major LLM API (OpenAI, Anthropic, Google) exposes a `temperature` parameter you can set directly. Lower it for factual Q&A or code generation where you want consistency; raise it for brainstorming or creative writing where variety helps. It's also the direct, mathematical explanation for why asking a model the exact same question twice can produce two different (but both perfectly plausible) answers.

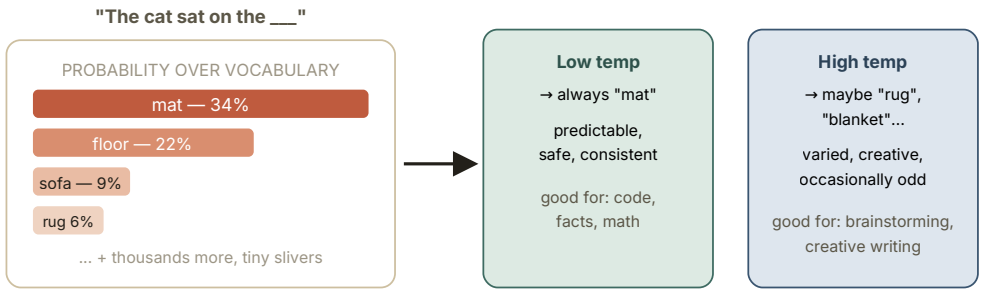


Fig 6.1 — Same probability distribution, two different temperature settings, two different personalities

6.3 How the model learned to do any of this

Everything above describes what a *trained* model does. Training is how it got there in the first place, and while the full mathematics is beyond this guide's scope, the core loop is genuinely simple to describe.

COMPLEX TERM, EXPLAINED

Training (Loss Function & Backpropagation)

In plain words: show the model a huge amount of real text with the final word hidden, let it guess, measure how wrong the guess's probabilities were, then nudge every internal number very slightly in the direction that would have made it less wrong – and repeat this billions of times.

The "how wrong" measurement is called the **loss function** — for language models, almost always **cross-entropy loss**, which penalizes the model more the further its predicted probability was from 100% on the actual correct word. **Backpropagation** is the algorithm that efficiently calculates exactly how much to adjust each of the model's billions of internal numbers (its "weights") to reduce that loss slightly, using calculus (specifically, the chain rule) — you don't need to run this math yourself, just know it's how credit and blame for a wrong prediction get assigned backward through every layer of the network.

◎ INTERVIEW SPOTLIGHT

"How is a language model trained?" — the concise, correct answer: **self-supervised next-token prediction**. The training text itself provides the "correct answer" for free — you just hide the next word and ask the model to guess it. This is why training data can be "the entire internet" rather than requiring humans to hand-label anything, which is the real reason these models could be scaled up so dramatically.

✦ WHY IT MATTERS TODAY

That basic loop — predict next word, measure error, adjust — only gets you a model that completes text plausibly. It does **not**, by itself, produce a model that's helpful, honest, and follows instructions the way Claude or ChatGPT do. That extra behavior comes from further training stages layered on top, commonly instruction fine-tuning and reinforcement learning from human feedback (RLHF) or related techniques — worth knowing exists, even without the details, since "why doesn't the base model behave like a chatbot" is a real and common point of confusion.

CHAPTER 07

Why Transformers Changed Everything

Zooming back out: why this architecture, and why now

7.1 Parallelization: the hardware fit

We covered this briefly in Chapter 1, but it's worth stating plainly: Transformers arrived at almost exactly the moment GPUs (and later, specialized chips like Google's TPUs) became widely available for training huge models. GPUs are built to do enormous numbers of simple calculations *simultaneously*. RNNs, being fundamentally sequential, couldn't take full advantage of that hardware. Transformers, being fundamentally parallel, were a near-perfect fit.

7.2 Scaling laws: bigger reliably means better

COMPLEX TERM, EXPLAINED

Scaling Laws

In plain words: a discovered, fairly predictable pattern where making a Transformer bigger, feeding it more data, and giving it more compute reliably makes it perform better – with no clear ceiling found yet at the sizes tried so far.

This mattered enormously in practice: it turned "build a better language model" from an uncertain research gamble into a comparatively predictable engineering bet — labs could estimate, in advance, how much better a model would get for a given increase in size, data, and compute. This predictability is a huge part of why massive investment poured into scaling these models up through the early 2020s.

✦ WHY IT MATTERS TODAY

This is the underlying story behind headlines like "Model X has 70 billion parameters" or "trained on N trillion tokens." **Parameters** are simply the count of all those learnable internal numbers (weights) across every attention and feed-forward layer in the model — GPT-3 had 175 billion; modern frontier models are believed to be substantially larger still (exact figures for many current models aren't publicly disclosed). Knowing that "parameter count" is a direct proxy for "model capacity," and that it's tightly connected to scaling laws, will make almost any AI news article make more sense.

7.3 One architecture, almost the entire field

What's remarkable, in hindsight, is how far a single core idea — let every element look at every other element and weigh relevance dynamically — generalized far beyond text.

DOMAIN	HOW ATTENTION GETS USED	EXAMPLES
Text generation	Words attending to other words	GPT, Claude, Gemini, LLaMA
Images	Image patches attending to other patches	Vision Transformer (ViT), parts of Stable Diffusion / Sora
Biology	Amino acids attending to other amino acids	AlphaFold (protein structure prediction)
Audio / Speech	Audio segments attending to other segments	Whisper and other speech models
Code	Tokens of code attending to other tokens	GitHub Copilot, Claude Code

INTERVIEW SPOTLIGHT

Mentioning that attention generalizes beyond text — for example, Vision Transformers treating an image as a grid of patches instead of a sentence of words — is a strong signal in an interview that you understand the *mechanism* itself, not just "the thing GPT uses." It shows you grasp attention as a general-purpose tool for relating elements of any sequence, not a language-specific trick.

CHAPTER 08

End-To-End Walkthrough

Every chapter above, followed through in a single continuous example

Let's trace one concrete example, start to finish, tying every chapter together. Suppose you type this into a decoder-only chat model (like Claude):

"The Eiffel Tower is in"

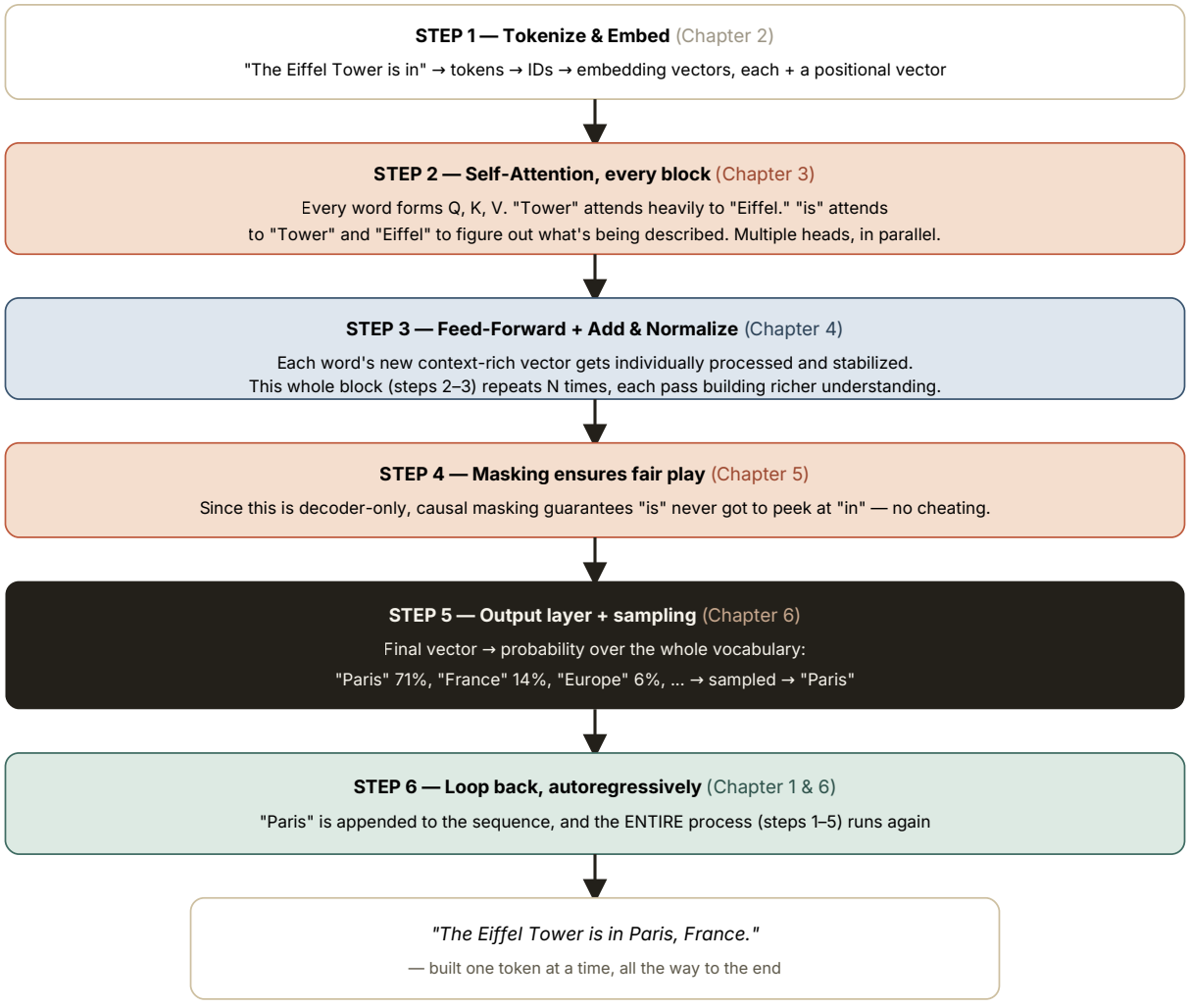


Fig 8.1 — One full pass through everything you've now learned, start to finish

Notice something important: the model didn't "know" the Eiffel Tower is in Paris the way a lookup table would. It generated "Paris" because, across its training data, the patterns of attention and the

learned weights made "Paris" overwhelmingly the most probable next token after that specific context. That distinction — **probabilistic pattern completion, not database lookup** — is the single most important mental model to carry forward from this entire guide.

CHAPTER 09

Quick-Reference Glossary

Every term from this guide, in one place, one line each

Attention — a mechanism letting every element of a sequence directly weigh the relevance of every other element, instead of relying on step-by-step memory.

Autoregressive Generation — producing output one token at a time, each new token conditioned only on the tokens generated so far.

Backpropagation — the algorithm that calculates how to adjust every internal weight to reduce prediction error, working backward through the network.

Causal Masking — forcibly hiding future tokens from the decoder's attention so it can't "see ahead."

Cross-Attention — attention where Queries come from one sequence (the decoder) and Keys/Values come from another (the encoder).

Decoder — the Transformer stack responsible for generating output, one token at a time, using masked self-attention.

Decoder-only Model — an architecture using only the decoder stack; the family GPT, Claude, LLaMA, and Gemini belong to.

Embedding — a vector of numbers representing a token's meaning as a point in a high-dimensional space.

Encoder — the Transformer stack responsible for reading and understanding a complete input sequence, with unrestricted attention.

Encoder-only Model — an architecture using only the encoder stack, suited to understanding/classification tasks (e.g. BERT).

Feed-Forward Network (FFN) — a small neural network applied independently to each token's vector, adding processing capacity between attention steps.

Key (K) — the vector a token uses to "advertise" what it contains, compared against Queries.

Layer Normalization — rescaling vectors after each step to keep their numerical range stable throughout a deep network.

LSTM / RNN — pre-Transformer sequence models that process input one step at a time, carrying a memory state forward.

Multi-Head Attention — running several attention calculations in parallel, each free to specialize in a different kind of relationship.

Parameters — the total count of learnable internal numbers (weights) in a model; a rough proxy for model size/capacity.

Positional Encoding — a vector added to each token's embedding to encode its position in the sequence.

Query (Q) — the vector a token uses to "ask" what it's looking for, compared against Keys.

Residual Connection — adding a layer's output back onto its original input rather than fully replacing it, easing training of deep networks.

RLHF — Reinforcement Learning from Human Feedback; further training that shapes a base model into a helpful, well-behaved assistant.

RoPE — Rotary Positional Embeddings; a modern positional encoding method used in many current LLMs.

Scaled Dot-Product Attention — the specific attention formula used in Transformers: similarity scores, scaled, softmaxed, then applied to Values.

Scaling Laws — the observed pattern that bigger models, more data, and more compute reliably produce better performance.

Self-Attention — attention where Queries, Keys, and Values all come from the same sequence.

Softmax — a function converting a list of raw numbers into positive values that sum to 100%, usable as weights or probabilities.

Temperature / Top-k / Top-p — settings controlling how randomly (vs. predictably) the model samples its next word from the probability distribution.

Token — a chunk of text (often a subword piece) that is the basic unit a language model reads and generates.

Tokenization — the process of splitting raw text into tokens, typically via a subword method like BPE.

Transformer Block / Layer — one repeatable unit combining multi-head attention, residuals, normalization, and a feed-forward network.

Value (V) — the vector a token hands over as its actual content once it's been selected as relevant by attention.

CHAPTER 10

Interview Question Bank

Real recurring questions, answered briefly – full detail is back in Chapters 1–8

Q Why did Transformers replace RNNs/LSTMs for most sequence tasks?

Parallelization (every token processed simultaneously, not step-by-step) and better handling of long-range dependencies via direct attention, avoiding the "forgetting" problem of recurrent memory.

Q Write the self-attention formula and explain each term.

Attention(Q,K,V) = softmax($QK^T/\sqrt{d_k}$)V. QK^T computes similarity between Queries and Keys; dividing by $\sqrt{d_k}$ keeps scores stable; softmax turns scores into weights summing to 1; multiplying by V blends the actual content accordingly.

Q Why scale by $\sqrt{d_k}$ specifically?

As vector dimensionality grows, raw dot products grow larger in magnitude, pushing softmax toward extreme, near one-hot outputs and causing unstable gradients. Dividing by the square root of the key dimension counteracts this growth.

Q What's the difference between self-attention and cross-attention?

In self-attention, Q, K, and V all come from the same sequence. In cross-attention, Queries come from one sequence (e.g. the decoder) while Keys and Values come from another (e.g. the encoder's output).

Q Why use multiple attention heads instead of one large attention operation?

Each head can specialize in a different type of relationship (syntax, coreference, semantics, etc.) rather than being forced to average everything into one generic notion of relevance.

Q Why do Transformers need positional encoding at all?

Because self-attention has no inherent sense of order — it treats input as an unordered set unless position information is explicitly injected into each token's vector.

Q What problem do residual (skip) connections solve?

They preserve a direct path for the original signal and gradient to flow through very deep networks, making deep architectures trainable and mitigating vanishing-gradient-type issues.

Q What does layer normalization do, and why is it needed?

It rescales each vector to keep values in a stable numerical range after each sub-layer, preventing activations from drifting to extreme scales as they pass through many stacked layers.

Q Explain the role of the feed-forward network inside a Transformer block.

It processes each token's vector independently (no cross-token interaction), adding non-linear representational capacity after attention has mixed in context from other tokens.

Q What is causal (masked) self-attention, and where is it used?

It blocks each position from attending to future positions by setting their scores to negative infinity before softmax. Used in decoders/decoder-only models to enforce left-to-right, autoregressive generation.

Q Is GPT/Claude an encoder, a decoder, or both?

Decoder-only. There is no separate encoder or cross-attention step — one masked self-attention stack handles both reading the conversation and generating the response.

Q What's the practical difference between BERT-style and GPT-style models?

BERT (encoder-only) sees the whole input at once and is suited to understanding/classification tasks. GPT-style (decoder-only) generates sequentially and is suited to open-ended text generation.

Q What is tokenization, and why not just split on whitespace into whole words?

Tokenization splits text into subword units (e.g. via Byte Pair Encoding). Whole-word tokenization would require a huge vocabulary and still fail on unseen words; subword tokenization guarantees any string can be represented, falling back to smaller pieces or characters.

Q What is an embedding, and what does "closeness" in embedding space mean?

A learned vector representing a token's meaning. Tokens used in similar contexts end up with vectors that are close together (by measures like cosine similarity), capturing semantic relationships.

Q How does temperature affect text generation?

It reshapes the probability distribution before sampling. Lower temperature sharpens it toward the single most likely token (deterministic, safe); higher temperature flattens it, giving less-likely tokens more chance of being picked (more varied, more error-prone).

Q What loss function is used to train language models, and why?

Cross-entropy loss, which penalizes the model based on how far its predicted probability for the actual correct next token was from 100%. It pairs naturally with softmax outputs.

Q What is self-supervised learning in the context of LLM pretraining?

Training where the "label" (the correct next token) comes for free from the text itself, with no human annotation required — this is what allowed pretraining on massive, largely unlabeled internet-scale text.

Q What does RLHF add on top of a base pretrained model?

Additional training (using human preference feedback, typically via reinforcement learning) that shapes a raw next-token predictor into a model that follows instructions and behaves as a helpful, aligned assistant.

Q What are scaling laws, and why did they matter for the field?

Empirically observed, fairly predictable relationships where more parameters, more data, and more compute reliably improve performance — turning "build a better model" into a more predictable engineering bet, which drove massive investment in scaling.

Q Does attention only apply to text?

No — the same mechanism generalizes to images (Vision Transformers, treating patches as tokens), audio, protein sequences (AlphaFold), and more, since it's fundamentally about relating elements of any sequence or set.

Q What is the time/space complexity concern with self-attention, and why does it matter?

Standard self-attention compares every token to every other token, so cost grows quadratically with sequence length ($O(n^2)$). This is a key reason handling very long documents/contexts is computationally expensive, and a major area of ongoing research (e.g. sparse or linear attention variants).

Q What's the difference between a model's context window and its parameter count?

Parameter count is the amount of learned "knowledge capacity" baked into the weights during training. Context window is how many tokens of conversation/input the model can attend to at once during a single use — a separate, architectural limit unrelated to how much the model "knows."

CHAPTER 11

The One-Page Cheat Sheet

Everything above, compressed into a single glance

THE ONE FORMULA THAT MATTERS MOST

$$\text{Attention}(Q, K, V) = \text{softmax}\left(Q \cdot K^T / \sqrt{d_k} \right) \cdot V$$

The Big Idea

Every word looks at every other word simultaneously, and learns how much to weigh each one — instead of reading sequentially with a fading memory.

The Pipeline

Text → tokens → embeddings + position → (attention → feed-forward) × N → output probabilities → sampled next token → repeat.

Q, K, V in one line

Query = what I'm looking for. Key = what I contain. Value = what I actually hand over once picked.

Why "multi-head"

Run attention several times in parallel so different heads can specialize in different relationships (grammar, reference, meaning...).

Residuals + LayerNorm

Residuals keep the original signal alive through deep stacks; normalization keeps the numbers stable. Together, they make deep training possible.

Masking

Decoders block attention to future tokens, forcing strictly left-to-right, autoregressive generation with no "cheating."

Three model families

Encoder-only (BERT, understanding) · Decoder-only (GPT/Claude, generation) · Encoder-Decoder (T5, translation).

Claude / GPT in one line

Decoder-only Transformers doing autoregressive next-token prediction over the whole conversation, trained first to predict text, then refined (e.g. via RLHF) to be a helpful assistant.

✦ If you remember only five things

1. **Attention lets every token directly weigh every other token** — this single mechanism is the reason Transformers work.

2. **Q, K, V are three learned projections of the same vector**, playing the roles of "what I want," "what I offer," and "what I give."
3. **Residuals + normalization** are what make it possible to stack this mechanism dozens or hundreds of times without training collapsing.
4. **Masking is what makes generation left-to-right and honest** — no peeking at tokens that don't exist yet.
5. **Claude, GPT, and friends are decoder-only** — one stack, reading and generating in the same breath, one token at a time.

✦ LAST THING WORTH KNOWING

None of this architecture explains *why* a model says any particular thing is true or gives any particular opinion — that behavior comes from training data and further fine-tuning, layered on top of this math. Understanding the architecture tells you *how* the machine works. It's a great foundation for reasoning critically about what these systems can and can't be expected to do well.